

코드쌤: 지혜야, 슬기야! 우리가 지난 시간에는 '클래스'라는 설계도를 사용해서 '강아지' 같은 객체들을 만들고, '상속'을 통해 더 특별한 강아지도 만들어봤었지?

지혜: 네! 클래스로 틀을 만들고, 그 틀에서 실제 강아지들을 찍어내듯이 만들 수 있어서 신기했어요! 상속으로 능력도 물려받고요! 🐶✨

슬기: 맞아요! 근데 선생님, Pygame에서 게임 캐릭터나 아이템 같은 것들을 만들 때도 클래스를 유용하게 쓸 수 있을 것 같아요! 개네들도 다 각자의 모양(이미지), 위치, 움직이는 방법 같은 정보랑 행동이 있잖아요.

코드쌤: 오호! 슬기가 아주 정확하게 핵심을 짚었네! 맞아. Pygame에서는 게임에 등장하는 다양한 요소들, 예를 들어 플레이어 캐릭터, 적, 총알, 아이템 등을 클래스로 만들어서 관리하면 정말 편리해. 그리고 Pygame은 이런 게임 요소들을 더 쉽게 다룰 수 있도록 특별한 도구를 제공하는데, 그게 바로 '스프라이트(Sprite)'와 '스프라이트 그룹(Sprite Group)'이란다!

코드쌤: '스프라이트'는 게임 화면에 나타나는 하나의 독립적인 객체(캐릭터, 아이템 등)를 의미한다고 생각하면 돼. 그리고 '스프라이트 그룹'은 이런 스프라이트들을 여러 개 담을 수 있는 '팀 가방' 같은 거야. 이 팀 가방에 스프라이트들을 넣어두면, 한꺼번에 이동시키거나, 한꺼번에 화면에 그리거나, 충돌 검사도 쉽게 할 수 있어!

(코드 조각 1: 'Player' 스프라이트 클래스 만들기 - 게임 캐릭터에게 유니폼을!)

```
import pygame # random 모듈은 아직 안 쓰이지만, 습관적으로 넣어둘 수도 있겠네요!
```

```
pygame.init()
```

```
WINDOW_WIDTH = 1200 # 화면이 더 넓어졌어요!
```

```
WINDOW_HEIGHT = 700
```

```
window_surface = pygame.display.set_mode((WINDOW_WIDTH, WINDOW_HEIGHT))
```

```
pygame.display.set_caption("몬스터 잡기 대작전 시작!") # 캡션 변경!
```

```
FPS = 60
```

```
clock = pygame.time.Clock()
```

```
# --- 여기가 오늘의 핵심! Player 캐릭터를 Sprite로 만들어보자! ---
```

```
# pygame.sprite.Sprite 클래스를 "상속" 받아서 Player 클래스를 만들 거야!
```

```
# 이렇게 하면 Player 클래스는 Pygame이 제공하는 Sprite의 특별한 능력들을 물려받게 돼!
```

```
class Player(pygame.sprite.Sprite): # 괄호 안에 (pygame.sprite.Sprite) 라고 써줬네!
```

```
    def __init__(self):
```

```
        # 부모 클래스(pygame.sprite.Sprite)의 __init__ 메소드를 먼저 실행시켜줘! (필수!)
```

```
        super().__init__()
```

```
        # Sprite가 되려면 꼭 필요한 두 가지!
```

```
        # 1. self.image : 화면에 보여줄 이미지 (Surface 객체)
```

```

self.image = pygame.image.load("hero.png") # 우리 히어로 이미지!
# 2. self.rect : 이미지의 위치와 크기를 담는 사각형 (Rect 객체)
self.rect = self.image.get_rect()

# 플레이어의 처음 위치를 정해주자!
self.rect.centerx = WINDOW_WIDTH/2 # 화면 가로 중앙
self.rect.bottom = WINDOW_HEIGHT # 화면 맨 아래쪽

self.velocity = 8 # 플레이어의 이동 속도!

# update 메소드: 매 프레임마다 호출되어서 플레이어의 상태를 업데이트하는 역할!
# Sprite Group에 속한 스프라이트들은 Group의 update()가 호출될 때, 각자의 update()가 자동으로 실행돼!
def update(self):
    keys = pygame.key.get_pressed() # 현재 눌린 키 정보 가져오기

    # 키 입력에 따라 플레이어 위치 이동 (화면 경계 처리 포함!)
    if keys[pygame.K_LEFT] and self.rect.left > 0:
        self.rect.x -= self.velocity
    # (원래 코드에는 elif가 아닌 if로 되어있어서, 여러 키를 동시에 누르면 이상하게 움직일 수 있어요.
    # 설명은 원래 코드대로 하되, 이런 점을 아이들이 발견하게 유도해도 좋아요!)
    if keys[pygame.K_RIGHT] and self.rect.right < WINDOW_WIDTH: # elif가 아니라 if!
        self.rect.x += self.velocity
    if keys[pygame.K_UP] and self.rect.top > 0: # elif가 아니라 if!
        self.rect.y -= self.velocity
    if keys[pygame.K_DOWN] and self.rect.bottom < WINDOW_HEIGHT: # elif가 아니라 if!
        self.rect.y += self.velocity

class Monster(): # 몬스터 클래스는 아직 아무 내용이 없네! (pass는 "아무것도 안함" 이라는 뜻)
    pass

# --- 플레이어 스프라이트를 만들고, 스프라이트 그룹(팀 가방)에 넣어주자! ---
# 1. 플레이어들을 담은 '팀 가방' 만들기!
my_player_group = pygame.sprite.Group()
# 2. Player 클래스(설계도)로 실제 플레이어(my_player) 만들기!
my_player = Player()
# 3. 만들어진 플레이어를 '팀 가방'에 쏙! 넣기!
my_player_group.add(my_player)

```

... (게임 루프 코드는 아래에) ...

지혜: class Player(pygame.sprite.Sprite): 이렇게 pygame.sprite.Sprite를 상속받았네요! 그럼 Player 클래스는 이제 그냥 Player가 아니라 '스프라이트 플레이어'가 되는 건가요?

코드쌤: 정확해, 지혜! pygame.sprite.Sprite를 상속받는 순간, 우리 Player 클래스는 Pygame이 제공하는 스프라이트만의 특별한 능력들을 물려받게 돼. 마치 슈퍼히어로 유니폼을 입는 것과 같지! 그리고 스프라이트가 되기 위해서는 두 가지 중요한 준비물이 꼭 필요해. 바로 self.image와 self.rect야.

슬기: self.image는 화면에 보여줄 그림이고, self.rect는 그 그림의 위치랑 크기 정보인 사각형이군요! 우리가 전에도 계속 썼던 것들이네요!

코드쌤: 맞아! 스프라이트는 기본적으로 "어떤 그림을(image), 어디에(rect)" 그릴지 알고 있어야 해. 그리고 Player 클래스 안에 update()라는 메소드가 있는데, 이건 아주 특별한 역할을 해.

지혜: update() 메소드요? 이건 언제 실행되는 거예요?

코드쌤: Player 스프라이트가 my_player_group이라는 '팀 가방(Group)'에 담겨있지? 나중에 게임 루프 안에서 우리가 my_player_group.update() 이렇게 팀 가방 전체에 "업데이트 해!" 라고 명령을 내리면, 팀 가방 안에 있는 모든 스프라이트들이 각자 가지고 있는 update() 메소드를 알아서 한 번씩 실행한다! 그래서 Player의 update() 안에는 플레이어의 키보드 입력에 따른 움직임 로직이 들어있는 거야. 매 프레임마다 "혹시 키 눌렀니? 그럼 움직여야지!" 하고 스스로 상태를 점검하고 바꾸는 거지.

슬기: 와! 그럼 Player 클래스 안에 update()만 잘 만들어두면, 나중에 플레이어가 많아져도 my_player_group.update() 한 줄만 쓰면 모든 플레이어가 알아서 움직이겠네요! 완전 편하겠다!

코드쌤: 빙고! 그게 바로 스프라이트 그룹을 사용하는 큰 장점 중 하나야! 자, 그럼 이제 게임 루프 안에서 이 스프라이트 그룹이 어떻게 활약하는지 볼까?

(코드 조각 2: 게임 루프에서 스프라이트 그룹 활용하기 - 팀워크 발휘!)

... (이전 코드는 그대로) ...

```
running = True
while running:
    for event in pygame.event.get(): # 이벤트 처리 (창 닫기 등)
        if event.type == pygame.QUIT:
            running = False

    # --- 화면 그리기 ---
    window_surface.fill((0, 0, 0)) # 1. 배경을 깨끗하게 지우고!

    # --- 스프라이트 그룹의 마법! ---
    # 2. "플레이어 팀 가방아, 너희 팀원들 전부 상태 업데이트 해!"
```

```
my_player_group.update() # 이 한 줄이면 my_player의 update()가 자동으로 실행!  
# 3. "플레이어 팀 가방아, 너희 팀원들 전부 화면에 그려줘!"  
my_player_group.draw(window_surface) # 이 한 줄이면 my_player가 화면에 켜!
```

```
pygame.display.update() # 4. 최종 완성된 화면을 보여줘!  
clock.tick(FPS) # 5. 시간아, 흘러라! (일정하게)
```

```
pygame.quit()
```

지혜: 와! 게임 루프가 엄청 간단해졌어요! `my_player_group.update()` 하니까 플레이어가 알아서 움직이고, `my_player_group.draw(window_surface)` 하니까 플레이어가 화면에 그려져요!

코드샴: 맞아! 만약 플레이어가 한 명이 아니라 여러 명이라고 해도, 저 두 줄은 그대로야. 팀 가방(Group)이 알아서 그 안에 있는 모든 스프라이트들에게 “업데이트 해!”, “화면에 나타나!” 하고 명령을 전달해주거든. 정말 편리하지?

슬기: Monster 클래스는 아직 pass라고 되어있는데, 나중에 저것도 `pygame.sprite.Sprite`를 상속받아서 만들고, 몬스터들을 담는 `my_monster_group` 같은 걸 또 만들 수도 있겠네요!

코드샴: 아주 훌륭한 생각이야, 슬기! 그렇게 되면 플레이어 그룹과 몬스터 그룹 사이의 충돌 검사 같은 것도 `pygame.sprite.groupcollide()` 같은 그룹 전용 마법으로 훨씬 쉽게 할 수 있단다! (이건 다음에 더 자세히!)

지혜: Player 클래스의 `update` 메소드 안에 있는 키보드 입력 처리 부분을 보니까, `elif`가 아니라 그냥 `if`로만 되어 있는데, 그럼 만약에 왼쪽 키랑 오른쪽 키를 동시에 누르면 어떻게 돼요?

코드샴: 오, 지혜가 아주 예리한 부분을 발견했네! 맞아. 지금 코드처럼 `if`로만 되어 있으면, 왼쪽 키와 오른쪽 키를 동시에 누르면 왼쪽으로 갔다가 바로 다시 오른쪽으로 가는 것처럼 보이거나, 아니면 거의 움직이지 않는 것처럼 보일 수도 있어. (실제로는 아주 짧은 시간 동안 왼쪽, 오른쪽 이동이 번갈아 일어날 수 있지.) 만약 한 번에 하나의 방향으로만 움직이게 하고 싶다면 `elif`를 사용하는 게 더 좋고, 대각선 이동을 허용하고 싶다면 또 다른 방식의 로직이 필요하겠지? 이런 것들을 직접 실험해보고 고쳐나가는 게 바로 코딩의 재미란다!

슬기: 스프라이트랑 그룹, 뭔가 게임 캐릭터들을 관리하는 데 정말 유용한 도구 같아요! 앞으로 게임 만들 때 꼭 써봐야겠어요!

코드샴: 아주 좋아! 스프라이트와 그룹은 Pygame으로 게임을 만들 때 코드를 훨씬 깔끔하고 효율적으로 만들어주는 핵심 기능 중 하나야. 오늘 배운 내용을 바탕으로 Monster 스프라이트도 한번 만들어보고, 여러 마리의 몬스터를 그룹에 담아서 화면에 나타나게 해보는 건 어떨까?